



Topic 2: Arrays

Jaber Taheri-Shakib

Research associate

Civil and Environmental Engineering

University of Waterloo | McMaster University

Sorting an Array in MATLAB

Function: `sort()`

Key idea

`sort()` rearranges array values in values in order.

Default: ascending order
smallest → largest

Use mode = 'descend' for
largest → smallest

Simple MATLAB examples

```
x = [7 2 9 4];
```

```
y = sort(x)
```

```
y = [2 4 7 9]
```

```
y = sort(x, 'descend')
```

```
y = [9 7 4 2]
```

```
[y, order] = sort(x)
```

```
y      = [2 4 7 9]
```

```
order = [2 4 1 3]
```

How the second output works

order tells the original position of each sorted value.

Original array

Value	7	2	9	4
Position	1	2	3	4

`sort(x)` ↓

After `sort(x)`

Value	2	4	7	9
Original position	2	4	1	3

Takeaway: sorted values = [2 4 7 9] and order = [2 4 1 3]

Using the Sort Function

```
1 % Vector of Forces [kN]
2 force = [100 50 75 33 25 200];
3
4 % MatLab's Default Sort
5 fprintf("\nMatLab's default sorting:\n")
6
7 ascendList = sort(force)
8 [~,ascendInd] = sort(force)
9
10 % Descending Sort
11 fprintf("\n A descending sort: \n")
12
13 [descendList, descendInd] = ...
14     sort(force, "descend")
```

MatLab's default sorting:

ascendList =

25	33	50	75	100	200
----	----	----	----	-----	-----

ascendInd =

5	4	2	3	1	6
---	---	---	---	---	---

A descending sort:

descendList =

200	100	75	50	33	25
-----	-----	----	----	----	----

descendInd =

6	1	3	2	4	5
---	---	---	---	---	---

Water Quality Data Processing in MATLAB

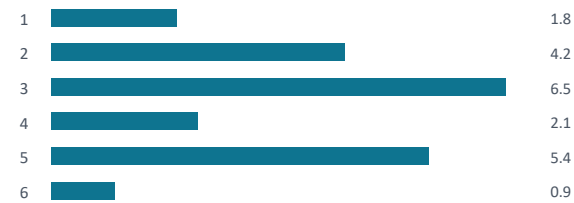
Topic 2: Arrays | Environmental engineering example

1 Monitoring data

Sample	pH	Chloride (mg/L)	Turbidity (NTU)
1	7.2	42	1.8
2	7.6	118	4.2
3	6.8	75	6.5
4	7.9	160	2.1
5	7.1	95	5.4
6	8.0	55	0.9

Each row is one water sample. Each column is one measured variable.

Turbidity values (NTU)



2 Ask MATLAB two questions

Question A
Which samples have the highest turbidity?

Use turbidity column = column 4
Sort high → low

Question B
Which samples have high chloride?

Use an example attention level:
chloride > 100 mg/L

MATLAB code idea

```
data = [sample pH chloride turbidity];  
  
sortedByTurb = sortrows(data,4,"descend");  
  
idx = data(:,3) > 100;  
highChloride = data(idx,:);
```

3 Read the results

After sorting by turbidity

- 1) Sample 3 → 6.5 NTU
- 2) Sample 5 → 5.4 NTU
- 3) Sample 2 → 4.2 NTU

After applying the condition

Sample 4: 160 > 100
Sample 2: 118 > 100

```
clc  
clear  
  
% Water-quality monitoring data  
sample = [1; 2; 3; 4; 5; 6];  
pH = [7.2; 7.6; 6.8; 7.9; 7.1; 8.0];  
chloride = [42; 118; 75; 160; 95; 55]; % mg/L  
turbidity = [1.8; 4.2; 6.5; 2.1; 5.4; 0.9]; % NTU  
  
% Combine all columns into one array  
data = [sample pH chloride turbidity];  
  
% Sort data by turbidity from highest to lowest  
sortedByTurb = sortrows(data,4,"descend");  
  
% Find samples with chloride greater than 100 mg/L  
idx = data(:,3) > 100;  
highChloride = data(idx,:);  
  
% Display results  
disp("Data sorted by turbidity:")  
disp(sortedByTurb)  
  
disp("Samples with chloride > 100 mg/L:")  
disp(highChloride)
```

Characters vs. Character Vectors

- A ***character*** is a single character in single quotes
- A character vector is sequences of characters in single quotes, e.g. 'hello and how are you?'
- **Every element in a character vector is a single character.**

```
charVec = 'MATLAB';  
  
numChar = numel(charVec);  
  
charSamp = charVec(3);  
charSet = charVec(1:3);  
  
phrase = [charVec ' is fun.']
```

Name	Value
charVec	'MATLAB'
numChar	6
charSamp	'T'
charSet	'MAT'
phrase	'MATLAB is fun.'

: Similar to numeric arrays, character arrays can be indexed, concatenated, modified, searched, and compared.

Character Vectors vs. Strings

```
charVec = 'MATLAB';  
  
numChar = numel(charVec);  
  
charSamp = charVec(3);  
charSet = charVec(1:3);  
  
phrase = [charVec ' is fun.']
```

Name	Value
charVec	'MATLAB'
numChar	6
charSamp	'T'
charSet	'MAT'
phrase	'MATLAB is fun.'

- Character vectors isolate each character as an individual element in an array, which can be accessed and manipulated.
- Strings lump all characters together in a single phrase (1x1 array), which must be treated as a whole unit (i.e. individual characters cannot be isolated).

```
strVec = "MATLAB";  
  
numStr= numel(strVec);  
  
charSamp = strVec(3);  
charSet = strVec(1:3);  
  
phrase = strVec + " is fun."
```

Name	Value
strVec	"MATLAB"
numStr	1
charSamp	Error
charSet	Error
phrase	"MATLAB is fun."

Comparing Phrases

Just like with numeric vectors, we can compare character vectors.

- Element-wise letter comparison for character arrays of equivalent length can be performed with relational operators. These comparisons work by type-casting the letters to their associated ASCII numeric values, and comparing those numbers. Element-wise comparisons cannot be achieved for string arrays.
- Whole-phrase comparisons for strings can be achieved using relational operators.
- Whole-phrase comparisons can also be achieved using the command `strcmp()`, which takes as input two character arrays or two strings.

```
name1 = 'Brad';  
name2 = 'Fred';  
name3 = 'Bob';  
name4 = 'Elizabeth';  
  
comp1 = name1 == name2;  
comp2 = strcmp(name1, name 2)  
comp3 = str(name1) == str(name2)  
comp4 = name3 == name4;  
comp5 = strcmp(name3, name 4)  
comp6 = str(name3) == str(name4)
```

Name	Value
name1	'Brad'
name2	'Fred'
name3	'Bob'
name4	'Elizabeth'
comp1	0 1 0 1

Name	Value
comp2	0
comp3	0
comp4	Error
comp5	0
comp6	0

Comparing Phrases

```
clc  
clear
```

```
name1 = 'Brad';  
name2 = 'Fred';  
name3 = 'Bob';  
name4 = 'Elizabeth';
```

```
comp1 = name1 == name2;  
comp2 = strcmp(name1, name2);  
comp3 = string(name1) == string(name2);
```

```
% This line gives an error because the character vectors  
% have different lengths.  
% comp4 = name3 == name4;
```

```
comp5 = strcmp(name3, name4);  
comp6 = string(name3) == string(name4);
```

```
disp(comp1)  
disp(comp2)  
disp(comp3)  
disp(comp5)  
disp(comp6)
```

```
1  clc  
2  clear  
3  
4  name1 = 'Brad';  
5  name2 = 'Fred';  
6  name3 = 'Bob';  
7  name4 = 'Elizabeth';  
8  
9  comp1 = name1 == name2;  
10 comp2 = strcmp(name1, name2);  
11 comp3 = string(name1) == string(name2);  
12  
13 % This line gives an error because the character vectors  
14 % have different lengths.  
15 % comp4 = name3 == name4;  
16  
17 comp5 = strcmp(name3, name4);  
18 comp6 = string(name3) == string(name4);  
19  
20 disp(comp1)  
21 disp(comp2)  
22 disp(comp3)  
23 disp(comp5)  
24 disp(comp6)
```

Command Window

```
0  1  0  1  
  
0  
  
0  
  
0  
  
0
```


Special Functions For Character Arrays

`strfind()`

- MatLab provides powerful built-in tools for searching character vectors and strings.
- MatLab's `strfind()` function will search a phrase for the existence of a specified substring and return the starting index of that substring (as if it were a character array).

`index = strfind(phrase, searchText)`

`% One Match`

```
phrase1 = 'Will this be on the exam?';  
index1 = strfind(phrase1, 'exam');
```

`% Multiple Matches`

```
phrase2 = 'dog cat bird dog dog';  
index2 = strfind(phrase2, 'dog');
```

`% No Matches`

```
phrase3 = 'Today is going to be a good day';  
search = 'week';  
index3 = strfind(phrase3, search);
```

Name	Value
phrase1	'Will this be on the exam?'
index1	21
phrase2	'dog cat bird dog dog'
index2	[1 14 18]
phrase3	'Today is going to be a good day'
search	'week'
index3	[]

Special Functions For Character Arrays

strrep()

MatLab's `strrep()` function performs a standard find and replace operation. The function takes in strings or character arrays, and returns output analogous to the respective input data type.

`newPhrase = strrep(originalPhrase, oldText, newText)`

```
1 fprintf('\n Original Character Vector\n')
2 phrase = 'I enjoy golf more than programming';
3 fprintf('%s\n', phrase)

4 phrase = strrep(phrase, 'golf', 'calculus')
5 fprintf('\nModified Character Vector\n')
6 fprintf('%s\n', phrase)
```

Original Character Vector
I enjoy golf more than programming

Modified Character Vector
I enjoy calculus more than programming

```
1 fprintf('\n Original Character Vector\n')
2 phrase = ['Thw tree on thw right was taller than'
3          ... 'thw tree on the left'];
4 fprintf('%s\n', phrase)

5 phrase = strrep(phrase, 'thw', 'the');
6 fprintf('\nModified Character Vector\n')
7 fprintf('%s\n', phrase)
```

Original Character Vector
Thw tree on thw right was taller than thw
tree on the left

Modified Character Vector
Thw tree on the right was taller than the
tree on the left

: The `strrep()`
function is case sensitive

Character Arrays

- Sometimes it can be advantageous to have an matrix of characters, such as a list of names. MatLab supports multi-dimensional character arrays, but generating them can be tricky.
- Let's try to list the three names 'Helena', 'Laura', and 'Amy' as a matrix in MatLab:

```
n1 = 'Helena';  
n2 = 'Laura';  
n3 = 'Amy';  
  
names = [n1; n2; n3]
```

**Error using vertcat
Dimensions of arrays being
concatenated are not consistent.**

: Vector dimensions must be compatible to be concatenated into an array. This is also valid for characters.

```
n1 = 'Helena';  
n2 = 'Laura';  
n3 = 'Amy';  
  
names = char(n1, n2, n3)
```

```
names =  
  
    3×6 char array  
  
    'Helena'  
    'Laura '  
    'Amy   '
```

: The `char()` function generates an array from disparate character vectors by padding the end of shorter character vectors with spaces.

String Arrays

Sometimes it's simpler to treat a word as a single variable, instead of as a set of characters, such as when you have a list of grocery items to read out. Setting the text as a string allows MatLab to register each phrase in " " a single value, which can be easily concatenated and read out.

```
g1 = "Lettuce";  
g2 = "Tomato";  
g3 = "Carrots";  
g4 = "Chicken";  
g5 = "Corn";  
g6 = "Salad Dressing";  
  
groceries = [g1; g2; g3; g4; g5; g6]
```

```
groceries =  
  
6x1 string array  
  
"Lettuce"  
"Tomato"  
"Carrots"  
"Chicken"  
"Corn"  
"Salad Dressing"
```

Searching String Arrays

Similar to numerical arrays, logical masking and the `find()` function can be used to search string arrays.

```
g1 = "Lettuce";  
g2 = "Tomato";  
g3 = "Carrots";  
g4 = "Chicken";  
g5 = "Corn";  
g6 = "Salad Dressing";  
  
groceries = [g1; g2; g3; g4; g5; g6];  
  
logMask1 = groceries == "Corn"  
logMask2 = groceries < "Corn"  
logInd = find(groceries == "Corn")
```

```
logMask1 =  
    6×1 logical array
```

```
0  
0  
0  
0  
1  
0
```

```
logMask2 =  
    6×1 logical array
```

```
0  
0  
1  
1  
0  
0
```

```
logInd =  
5
```

Other Helpful Functions for Phrases

Function	Description	Example	
		Script	Output Variable Value
<code>sort(strArray)</code>	returns a all elements of <code>strArray</code> sorted in alphabetical order	<pre>sort(["Hi"; "Class"; "Of"; "2027"])</pre>	<pre>["2027" "Class" "Hi" "Of"]</pre>
<code>upper(phrase)</code>	returns a character array or string in which all elements of the phrase are upper case	<code>upper('hello world!')</code>	<code>'HELLO WORLD!'</code>
<code>lower(phrase)</code>	returns a character array or string in which all elements of the phrase are lower case	<code>lower('Hello World!')</code>	<code>'hello world!'</code>
<code>isspace(charArray)</code>	returns a logical vector of the same length as <code>charArray</code> , in which the value 1 represents a corresponding element of ' ' in <code>charArray</code>	<code>isspace('Hello World!')</code>	<code>[0 0 0 0 0 1 0 0 0 0 0 0]</code>
<code>isletter(charArray)</code>	returns a logical vector of the same length as <code>charArray</code> , in which the value 1 represents that the corresponding element of <code>charArray</code> a letter of the alphabet.	<code>isletter('Hello World!')</code>	<code>[1 1 1 1 1 0 1 1 1 1 0]</code>

Example

The following table shows water-quality monitoring data from 8 samples. Each row is one sample. The numeric variables are pH, chloride concentration, and turbidity. The text variables are sample ID, station name, and field note.

Using MATLAB arrays, write a code to:

1. Combine the numeric data into one array called data.
2. Sort all samples by turbidity from highest to lowest.
3. Sort chloride values from highest to lowest and report the original sample order.
4. Find samples that need review using this condition:
 $\text{chloride} > 100 \mid \text{turbidity} > 5 \mid \text{pH} < 6.5$
5. Clean the field notes by replacing "clorid" with "chloride".
6. Find samples where the note contains "odor".
7. Find all samples collected from "East Basin".
8. Compare one character vector and one string to understand the difference between ' ' and " ".

Sample No.	Sample ID	Station Name	pH	Chloride mg/L	Turbidity NTU	Field Note
1	S01	North Gate	7.2	42	1.8	clear water
2	S02	East Basin	6.8	118	4.2	slightly cloudy
3	S03	West Creek	8.1	75	6.5	odor detected
4	S04	South Pond	7.5	160	2.1	chloride high
5	S05	Main Drain	6.4	95	5.4	very turbid
6	S06	East Basin	7.9	55	0.9	clear
7	S07	North Gate	7.0	132	7.3	ODOR and cloudy
8	S08	West Creek	8.3	88	3.6	typo: clorid high